CSC 345 — Exam 1 Study Guide

Banner — scope PROVISIONAL: Topics 1–3. The exam scope is **not published**. Practice covers the lectures so far:

- **Topic 1, Review:** lists, stacks, queues, linked lists, orthogonal lists, array storage, recursion.
- **Topic 2, Algorithm Analysis:** step-counting, profiling, asymptotic notation, recurrences.
- **Topic 3, Graphs:** representations, BFS/DFS, shortest paths, and the spanning-tree and topological-sort material in the Topic 3 slides.

Topic 4 (Internal Sort) slides are **not linked yet**, so Topic 4 is excluded. The **date is set**: Thu Oct 8, 2026; no clock time is published.

Canonical maps: `csc345-topic-map.md` · `announced-assessments-inventory-2026-09-24.md` · `site-audit-2026-09-24.md`

1. Header

Field	Value	Status
Course	CSC 345 Analysis of Discrete Structures (FA26), TuTh 2:00–3:15 PM, The Commons 305	Confirmed
Assessment	Exam 1	Confirmed
Date	**Thu Oct 8, 2026**	**Scheduled** (syllabus)
Time / format	Exam clock time **not published**. Format **not published**; the syllabus says in person, and Gradescope's role is unknown.	NV
Coverage	Topics 1–3	**PROVISIONAL**
Topic 3 depth	Topic 3 is scheduled for weeks 5–6 (week 6 has not happened yet). The slides include shortest paths, minimum-cost spanning trees (MCST), and topological sort. **How far lecture gets before Oct 8 is unverified.**	Watch lecture
Same day	CSC 335 Midterm, 12:30 PM class in the same room (clock time unverified)	Plan energy
Guide updated	2026-09-24 (America/Phoenix) — Opus review pass	

All times are **Arizona time (MST)**.

Sources

Source	Role
`Topic_1_Slides.pdf`, `Topic_2_Slides.pdf`, `Topic_3_Slides.pdf` (+ txt)	Topic content
`csc345_syllabus.pdf`	Exam date; quizzes (best 5 count, no make-ups)
Topic map, inventory, site audit	Topic lists per slide deck; provisional label
Gradescope format notes `csc345-2026-09-24.md`	Homework 1 rubric shape (Questions 1–8); score signals

---

2. Topic checklist

Order	Section ID	Topic	Blind ✓	Check ✓	Retake ✓
1	`345-E1-t1-lists`	Stacks, queues, singly/doubly/circular linked lists			
2	`345-E1-t1-storage`	Array storage (row-/column-major) and orthogonal lists			
3	`345-E1-t1-rec`	Recursion			
4	`345-E1-t2-count`	Step-counting and profiling			
5	`345-E1-t2-asym`	Big-O / Ω / Θ with witnesses			
6	`345-E1-t2-rec`	Recurrences and the Master Theorem			
7	`345-E1-t3-basics`	Graph definitions and representations			
8	`345-E1-t3-search`	BFS and DFS			
9	`345-E1-t3-paths`	Shortest paths (Dijkstra)			
10	`345-E1-t3-mcst-topo`	MCST (Prim/Kruskal) and topological sort — **only as far as lectured**			

---

3. Explanations + worked examples

Topic 1 — Lists, stacks, queues (`345-E1-t1-lists`)

- A **stack** is LIFO (push and pop at the top). A **queue** is FIFO (enqueue at the rear, dequeue at the front).
- **Singly linked list with only a head:** prepend is O(1), but appending at the end is O(n) because you must walk to the end. A **tail pointer** makes append O(1). Deleting the tail is still O(n) in a singly linked list, because you need the node before it.
- A **doubly linked list** can delete a known node in O(1).

- **Circular list** where `tail.next` is the front. To append `new`: set `new.next = tail.next` (the front), then `tail.next = new`, then `tail = new`.
- Python's `list` is a dynamic **array**, not a linked list (the slides make this point).

Topic 1 — Array storage and orthogonal lists (`345-E1-t1-storage`)

- A 2D array with R rows and C columns, element size s, and 0-based indices:
- **Row-major:** addr(i, j) = base + (i·C + j)·s
- **Column-major:** addr(i, j) = base + (j·R + i)·s
- An **orthogonal list** stores a sparse matrix as nodes for the nonzero entries only. Each node has (row, col, value) plus a **right** link (next nonzero in its row) and a **down** link (next nonzero in its column), with row and column header lists. Memory is O(R + C + nonzeros) instead of O(R·C).
- PP1 used this structure. This guide gives concepts only, with no project code.

Topic 1 — Recursion (`345-E1-t1-rec`)

- A recursive definition needs a **base case** and a **recursive case** that moves toward the base.
- To trace a recursive call, draw the call stack. Each call waits for its sub-calls to return.

Topic 2 — Step-counting and profiling (`345-E1-t2-count`)

- The **instance characteristic** is the input size, n.
- **Step-counting** counts basic operations as a function of n. **Profiling** measures real run time. Profiling needs working code and is noisy (other processes, caching, hardware), so the slides prefer counting and asymptotics for comparisons.
- **Nested loop pattern:** `for i = 1..n: for j = 1..i: step` does 1 + 2 + … + n = n(n+1)/2 steps, which is $\Theta(n^2)$.

Topic 2 — Asymptotic notation (`345-E1-t2-asym`)

- **f is O(g)** if there exist c > 0 and n_0 such that f(n) ≤ c·g(n) for all n ≥ n_0. Write "f is O(g)" or "f ∈ O(g)"; the slides warn against "f = O(g)".
- **Ω** is the lower-bound version (f(n) ≥ c·g(n)). **Θ** means both O and Ω.
- **A complete proof has four parts:** (1) state what you are proving; (2) choose c and n_0 explicitly; (3) show the inequality holds for every n ≥ n_0; (4) conclude. Homework 1 lost points that, per the score signal, likely relate to proof completeness. The rubric comments were not captured, so this is a likely cause, not a confirmed one.
- **Worked example:** 7n + 8 is O(n). For n ≥ 1, 7n + 8 ≤ 7n + 8n = 15n, so c = 15 and n_0 = 1.
- **Showing something is not O(g):** assume c and n_0 exist, then exhibit an n ≥ n_0 that breaks the inequality.

Topic 2 — Recurrences and the Master Theorem (`345-E1-t2-rec`)

For T(n) = a·T(n/b) + c·n^d with a ≥ 1, b > 1, d ≥ 0 (the form used in the slides):

- If a < b^d, then T(n) is $\Theta(n^d)$.
- If a = b^d, then T(n) is $\Theta(n^d \log n)$.
- If a > b^d, then T(n) is Θ(n^(log_b a)).

Worked example: $T(n) = 2T(n/2) + n$ has $a = 2$, $b = 2$, $d = 1$. Since $a = b^d$, $T(n)$ is $\Theta(n \log n)$.

Recurrences that do not match this form, such as $T(n) = T(n-1) + c$, can be solved by **unrolling**, and the result can be proved by **induction**.

Topic 3 — Graph basics (`345-E1-t3-basics`)

- $G = (V, E)$. Undirected edges are sets $\{u, v\}$; directed edges are ordered pairs (u, v).
- Key terms: adjacent, degree (for directed graphs, in-degree and out-degree), path, **simple path** (no repeated vertices), cycle, connected, DAG (directed acyclic graph).
- **Handshake lemma:** in an undirected graph, $\Sigma \deg(v) = 2|E|$.

Representation	Space	Is (u, v) an edge?	List the neighbors of u
Adjacency matrix	$\Theta(V^2)$	$O(1)$	$O(V)$
Adjacency list	$\Theta(V + E)$	$O(\deg u)$	$O(\deg u)$

Topic 3 — BFS and DFS (`345-E1-t3-search`)

- **BFS** uses a queue and visits vertices in layers by edge-distance. On an unweighted graph it finds shortest paths by edge count.
- **DFS** uses a stack or recursion and goes deep first.
- Both run in $O(V + E)$ with adjacency lists, or $O(V^2)$ with a matrix.
- If the graph is disconnected, restart the search from any vertex not yet visited.
- **Tie-break convention:** when the problem says "alphabetical order", always take neighbors in that order.

Topic 3 — Shortest paths (`345-E1-t3-paths`)

Dijkstra's algorithm (single source, non-negative weights):

1. Set dist(source) = 0 and every other distance to ∞.
2. Repeatedly **finalize** the unfinalized vertex with the smallest distance.
3. For each edge out of it, **relax**: if dist(u) + w < dist(v), set dist(v) = dist(u) + w.

Dijkstra fails when there are negative edge weights.

Topic 3 — MCST and topological sort (`345-E1-t3-mcst-topo`) — only as far as lectured

- **Prim:** grow one tree, adding the cheapest edge that leaves the tree each time.
- **Kruskal:** sort all edges by weight, and add each edge unless it would create a cycle (tracked with union-find).
- **Topological sort** orders a DAG so every edge goes from earlier to later. One method: repeatedly remove a vertex with in-degree 0. Another: order vertices by reverse DFS finish time.
- PP2 (due Oct 13) implements graph algorithms. This guide gives **hand traces only**, with no project code.

---

4. Practice questions (core)

P1 (recall). Which structure is FIFO, and which is LIFO?

P2 (solve). In a singly linked list with only a head pointer, what are the costs of appending at the end and of prepending?

P3 (trace). A circular list's tail points at the last node. Give the pointer steps to append X.

P4 (solve). Is $3n^2 + 2n$ in $O(n^2)$? Justify with c and n_0.

P5 (Master). Solve $T(n) = 4T(n/2) + n$ with the Master Theorem. State a, b, d, and the case.

P6 (recall). Why is wall-clock timing a weak sole measure of an algorithm?

P7 (define). Give V and E for the undirected square A–B–C–D–A.

P8 (BFS). Edges A–B, B–C, A–D. What is the BFS order from A, taking neighbors alphabetically?

P9 (DFS). Same graph. What is a DFS order from A, recursive, taking neighbors alphabetically?

P10 (explain). Which data structure drives BFS, and which drives DFS?

P11 (recall). When is Exam 1?

P12 (explain). Why is Topic 4 excluded from this guide?

4b. Gradescope-format practice — Practice recommendation

Where this format comes from. The Gradescope format notes show Homework 1 graded as rubric Questions 1–8 (plus a 0-point late-adjustment item) on a written PDF, and a Quiz 1 scored out of 10. The syllabus says quizzes are short, the best five count, and there are no make-ups. The **Exam 1 format is not published**. These items mirror the quiz grain and homework proof habit. They are not a claimed exam format.

Score signals, in words only (this copy lives in a public repository): the Quiz 1 score suggests tightening **definitions**, and the Homework 1 deductions suggest writing **complete proofs**.

Timed mock (Mon Oct 5): do Q1–Q8 in 15 minutes, then PR1–PR3 and TR1–TR4 in 45 minutes.

Quiz-grain concept checks (about 10 points total)

GS-345-Q1. In one sentence each, give the removal order of a stack and of a queue.

GS-345-Q2. True or false (fix it if false): "Python's `list` is a linked list."

GS-345-Q3. Give witnesses c and n_0 showing that $2n + 5$ is $O(n)$.

GS-345-Q4. For $T(n) = 2T(n/2) + n$, state a, b, d, and the Master case.

GS-345-Q5. Name the frontier data structure for BFS and for DFS.

GS-345-Q6. What is the difference between a path and a simple path?

GS-345-Q7. A 4×5 int array (4 bytes per element) is stored row-major at base 1000. What is the address of A[2][3]?

GS-345-Q8. Give the space cost of an adjacency matrix and of an adjacency list, in terms of V and E.

Proof write-ups (Homework-rubric style — write each as one complete answer)

GS-345-PR1. Prove that $5n^2 + 3n$ is $O(n^2)$. Use all four parts: statement, witnesses, inequality for all $n \geq n_0$, conclusion.

GS-345-PR2. Prove that n^2 is **not** $O(n)$.

GS-345-PR3 (induction habit). Let $T(1) = 1$ and $T(n) = T(n-1) + 2$ for $n > 1$. Prove by induction that $T(n) = 2n - 1$ for all $n \geq 1$.

GS-345-PR4. Prove the handshake lemma: $\Sigma \deg(v) = 2|E|$ for any undirected graph.

Traces and short solves

GS-345-TR1 (Master ×3). Solve each: (a) $T(n) = 8T(n/2) + n^2$; (b) $T(n) = T(n/2) + 1$; (c) $T(n) = 2T(n/4) + n$.

GS-345-TR2 (step count). Count exactly how many times `x++` runs, as a function of n:

```
for i = 1 to n
  for j = 1 to i
    x++
```

GS-345-TR3 (BFS/DFS with distances). The undirected edges are A–B, A–C, B–D, C–D, D–E, E–F. Take neighbors alphabetically.

- (a) Give the BFS order from A and each vertex's BFS distance.
- (b) Give the recursive DFS order from A.

GS-345-TR4 (Dijkstra). The directed weighted edges are S→A 4, S→B 1, B→A 2, A→C 1, B→C 5. Run Dijkstra from S. Give the order in which vertices are finalized and the final distances.

GS-345-TR5 (stack/queue trace). On an empty stack, run push 1, push 2, pop, push 3, push 4, pop, pop. What is popped, and what remains? Repeat with a queue (enqueue and dequeue).

GS-345-TR6 (only if lectured: Kruskal / topological sort).

- (a) Undirected weighted edges: A–B 3, A–C 1, B–C 2, B–D 4, C–D 5. List the edges Kruskal adds and the total weight.
- (b) DAG edges: A→B, A→C, B→D, C→D. Give one valid topological order.

5. ANSWER KEY

Core keys

A1. A queue is FIFO; a stack is LIFO.

A2. Prepend is O(1). Appending at the end is O(n) without a tail pointer, because you must walk the whole list.

A3. `X.next = tail.next` (the front), then `tail.next = X`, then `tail = X`.

A4. Yes. For $n \geq 1$, $3n^2 + 2n \leq 3n^2 + 2n^2 = 5n^2$, so $c = 5$ and $n_0 = 1$.

A5. $a = 4$, $b = 2$, $d = 1$, so $b^d = 2 < 4 = a$. This is the $a > b^d$ case: $\Theta(n^{(\log_2 4)}) = \Theta(n^2)$.

A6. It requires finished, debugged code. It mixes in noise from the operating system, caching, and hardware. And it measures one machine and one input, not growth rate. Operation counting and asymptotics avoid all three problems.

A7. V = {A, B, C, D}. E = {{A,B}, {B,C}, {C,D}, {D,A}}.

A8. A, B, D, C.

A9. A, B, C, D. From A go to B, then C; back up to A; then visit D.

A10. BFS uses a queue. DFS uses a stack (or the call stack when recursive).

A11. Thu Oct 8, 2026. No clock time is published.

A12. Topic 4 slides are not linked yet, and the provisional scope label is Topics 1–3.

Gradescope-format keys

GS-345-Q1. A stack removes the most recently added item first (LIFO). A queue removes the oldest item first (FIFO).

GS-345-Q2. False. It is a dynamic array.

GS-345-Q3. For $n \geq 1$, $2n + 5 \leq 2n + 5n = 7n$, so $c = 7$ and $n_0 = 1$.

GS-345-Q4. $a = 2$, $b = 2$, $d = 1$. Since $a = b^d$, $T(n)$ is $\Theta(n \log n)$.

GS-345-Q5. BFS uses a queue. DFS uses a stack (or recursion).

GS-345-Q6. A simple path never repeats a vertex. A general path may repeat vertices, unless the course definition restricts it.

GS-345-Q7. $1000 + (2 \cdot 5 + 3) \cdot 4 = 1000 + 52 =$ **1052**.

GS-345-Q8. Matrix: $\Theta(V^2)$. List: $\Theta(V + E)$.

GS-345-PR1.

1. **Claim:** there exist c, n_0 with $5n^2 + 3n \le c \cdot n^2$ for all $n \ge n_0$.
2. **Witnesses:** c = 8, $n_0 = 1$.
3. **Inequality:** for $n \ge 1$ we have $n \le n^2$, so $3n \le 3n^2$, and therefore $5n^2 + 3n \le 5n^2 + 3n^2 = 8n^2$.
4. **Conclusion:** $5n^2 + 3n$ is $O(n^2)$.

GS-345-PR2. Suppose $c > 0$ and n_0 exist with $n^2 \le c \cdot n$ for all $n \ge n_0$. Dividing by $n > 0$ gives $n \le c$ for all $n \ge n_0$. Take $n = \max(n_0, \lceil c \rceil + 1)$. Then $n \ge n_0$ but $n > c$, a contradiction. So n^2 is not $O(n)$.

GS-345-PR3.

- **Base case:** $T(1) = 1 = 2 \cdot 1 - 1$. ✓
- **Inductive hypothesis:** $T(k) = 2k - 1$ for some $k \ge 1$.
- **Inductive step:** $T(k+1) = T(k) + 2 = (2k - 1) + 2 = 2(k+1) - 1$. ✓
- **Conclusion:** by induction, $T(n) = 2n - 1$ for all $n \ge 1$.

GS-345-PR4. Each edge {u, v} adds exactly 1 to deg(u) and 1 to deg(v), so each edge contributes exactly 2 to the total $\Sigma \deg(v)$. Summing over all edges gives $2|E|$.

GS-345-TR1.

- (a) a = 8, b = 2, d = 2; $b^d = 4 < 8$, so $\Theta(n^{\wedge}(\log_2 8)) =$ **$\Theta(n^3)$**.
- (b) a = 1, b = 2, d = 0; $b^d = 1 = a$, so $\Theta(n^0 \log n) =$ **$\Theta(\log n)$**.
- (c) a = 2, b = 4, d = 1; $b^d = 4 > 2$, so **$\Theta(n)$**.

GS-345-TR2. $1 + 2 + \ldots + n =$ **$\frac{n(n+1)}{2}$**, which is $\Theta(n^2)$.

GS-345-TR3.

- (a) BFS order: A, B, C, D, E, F. Distances: A 0, B 1, C 1, D 2, E 3, F 4.
- (b) DFS order: A, B, D, C, E, F. From A go to B, then D. D's first unvisited neighbor is C, whose neighbors are all visited, so back up to D. Then visit E, then F.

GS-345-TR4.

- Finalized order: S (0), B (1), A (3, via B: 1 + 2 beats the direct 4), C (4, via A: 3 + 1 beats 1 + 5 = 6).
- Final distances: S 0, B 1, A 3, C 4.

GS-345-TR5.

- Stack: pops 2, 4, 3; remaining [1].
- Queue: dequeues 1, 2, 3; remaining [4].

GS-345-TR6.

- (a) Kruskal considers A–C 1 (add), B–C 2 (add), A–B 3 (skip, it would create a cycle), then B–D 4 (add). The total weight is 1 + 2 + 4 = **7**.
- (b) A, B, C, D (A, C, B, D is also valid).

---

6. Common mistakes + readiness

- Saying Python's `list` is a linked list.
- Forgetting the tail update on a circular append.
- Writing "f = O(g)", or skipping explicit witnesses c and n_0.
- Proofs missing a part, especially the conclusion line or "for all $n \geq n_0$".
- Misreading a, b, or d in the Master Theorem, or applying it to $T(n-1)$ forms.
- Running BFS or DFS on a disconnected graph without restarting.
- Mixing up row-major and column-major address formulas.
- Relaxing an edge in Dijkstra after its target is already finalized.

Readiness: a blind pass over all 10 IDs, the four-part proof habit written three times, redrawn BFS/DFS/Dijkstra traces, a light mixed review on Wednesday, and no cram the night before.

---

7. Day-by-day prep

Day	Focus	IDs
Fri Sep 25	Topic 1: lists, storage, recursion	t1-*
Sat Sep 26	Topic 2: counting, asymptotics, proofs	t2-count, t2-asym
Sun Sep 27	Topic 2: recurrences, Master Theorem	t2-rec
Fri Oct 2	Topic 3: basics, BFS, DFS	t3-basics, t3-search
Sat Oct 3	Topic 3: Dijkstra, plus MCST/topological sort if lectured	t3-paths, t3-mcst-topo
Mon Oct 5	**Timed mock** (§4b), then even rotation	all
Tue Oct 6	Optional light flash only (the 337 midterm is today)	—
Wed Oct 7	Light mixed review, not a cram	all
Thu Oct 8 morning	Calm buffer	—

The Tuesday/Thursday class days of Sep 29 and Oct 1 are left light. PP2 is due Tue Oct 13, after the exam.

---

8. Changes in this review (2026-09-24)

- **Filled Topic 1–3 gaps** from the site audit and topic map: orthogonal lists and array storage, recursion, step-counting vs profiling, Ω/Θ, shortest paths (Dijkstra), and MCST/topological sort (marked "only as far as lectured").
- Added four proof write-ups (PR1–PR4), including a "not O(n)" proof and an induction proof, since Homework 1 was induction proofs. Added traces TR1–TR6. All have keys.
- Restored the **syllabus** facts "best 5 quizzes count, no make-ups". These come from the syllabus, not Gradescope.
- Fixed the ambiguous "Fri Sep 25 / Oct 2" plan row. Every row now has one date.
- Described scores in words only, because the repository is public.
- Changed times to Arizona time.

9. Disclaimer

The Gradescope format notes describe Homework 1 and Quiz 1 only. The Exam 1 blueprint is **not published**, so §4b is a practice recommendation.

Study aid only — not for submitted work. This guide contains no PP2 code. Follow CSC 345 and UA policies.